

## 4 Export/Import : Organisation Modulaire

### 4.1 Pourquoi Exporter/Importer ?

**Problème :** Tout le code dans un seul fichier devient ingérable

**Solution :** Modules ES6 (fichiers séparés)

Listing 16 – Organisation modulaire

```
src/  
  components/  
    Header.jsx      // Export Header  
    Footer.jsx      // Export Footer  
    ProductCard.jsx // Export ProductCard  
  App.jsx           // Import tout
```

### 4.2 Export Default

Un seul export default par fichier.

Listing 17 – Export default - Syntaxe

```
// Button.jsx  
function Button({ label, onClick }) { // Props  
  return <button onClick={onClick}>{label}</button>;  
}  
  
export default Button;  
  
// OU en une ligne  
export default function Button({ label, onClick }) {  
  return <button onClick={onClick}>{label}</button>;  
}
```

**Import export default :**

Listing 18 – Import default

```
// App.jsx  
import Button from './components/Button';  
import Btn from './components/Button'; // Renommage libre  
import MonBouton from './components/Button'; // Autre nom  
  
function App() {  
  return <Button label="Cliquer" onClick={() => alert('OK')} />;  
}
```

#### ★ Astuce

**Caractéristiques export default :**

- Un seul par fichier
- Import sans accolades { }
- Nom import libre (renommage)

### 4.3 Named Export

Plusieurs exports nommés par fichier.

Listing 19 – Named export - Syntaxe

```
// utils.js
export const add = (a, b) => a + b;
export const multiply = (a, b) => a * b;
export const PI = 3.14159;

// OU groupe
const add = (a, b) => a + b;
const multiply = (a, b) => a * b;

export { add, multiply };
```

Import named export :

Listing 20 – Import named

```
// App.js
import { add, multiply } from './utils';

console.log(add(2, 3)); // 5
console.log(multiply(4, 5)); // 20

// Renommer avec 'as'
import { add as addition } from './utils';
console.log(addition(2, 3)); // 5

// Tout importer
import * as Utils from './utils';
console.log(Utils.add(2, 3)); // 5
console.log(Utils.multiply(4, 5)); // 20
```

#### △ Important

Différences clés :

| Export Default     | Named Export          |
|--------------------|-----------------------|
| 1 seul par fichier | Plusieurs par fichier |
| Import sans { }    | Import avec { }       |
| Nom libre          | Nom exact (ou as)     |
| import X from      | import { X } from     |

## 4.4 Barrel Exports

Les **barrel exports** (fichiers `index.js`) simplifient les imports multiples.

**Problème initial :**

Listing 21 – Imports verbeux

```
// App.jsx - Beaucoup d'imports
import Header from './components/Header/Header';
import Footer from './components/Footer/Footer';
import ProductCard from './components/ProductCard/ProductCard';
import Button from './components/Button/Button';
import Input from './components/Input/Input';
```

**Solution : Fichier `index.js`**

Listing 22 – Structure avec barrel

```
components/
  Header/
    Header.jsx
    Header.css
  Footer/
    Footer.jsx
    Footer.css
  ProductCard/
    ProductCard.jsx
    ProductCard.css
index.js ← Barrel export
```

**Contenu `index.js` :**

Listing 23 – `components/index.js`

```
// components/index.js
export { default as Header } from './Header/Header';
export { default as Footer } from './Footer/Footer';
export { default as ProductCard } from './ProductCard/ProductCard';
export { default as Button } from './Button/Button';
export { default as Input } from './Input/Input';
```

**Import simplifié :**

Listing 24 – Import avec barrel

```
// App.jsx - UN SEUL import
import { Header, Footer, ProductCard, Button, Input } from './components';

function App() {
  return (
    <div>
      <Header />
      <ProductCard name="Laptop" price={5000} inStock={true} />
      <Footer />
    </div>
  );
}
```

**Avantages :**

- Imports groupés et propres
- Un seul point d'entrée par dossier
- Refactoring facilité
- Code plus maintenable